

You are building a small slice of an **OT/ICS cybersecurity monitoring platform**. The product helps industrial sites (ports, plants, energy infrastructure) keep track of their operational technology assets, the vulnerabilities on those assets, and how resilient each site is against cyber risk.

In this role you sit between two worlds. On one side you translate what the CEO and other stakeholders want to see into a working web application. On the other side you take the raw data the data team puts in the database and turn it into something people can actually use. This assessment is designed to test both: your **technical engineering skills** and your **product/UX judgment**. We weigh them equally.

This is a take-home assignment scoped for roughly **4 hours**. We are not looking for a finished product. We are looking for how you think, what you prioritize, your creativity, and the quality of what you do build.

Must-have

1. Asset Inventory

The analyst should be able to find what he needs fast.

2. Asset Detail

Clicking an asset opens its detail view.

PICK ONE OF THESE TWO (YOUR CHOICE)

3a. Resilience Index Dashboard

A score-driven view of how the plant site is doing on its security controls.

3b. Overview Dashboard

A general overview of how the plant site is doing. Not every part of this needs to link necessary to a screen/view.

The data

We have four files with data that contains asset information, security controls (for resilience index) and vulnerabilities.

- `assets.ts`
- `security-controls.ts`
- `vulnerabilities.ts`
- `types.ts`

A little disclaimer: the data has been generated by AI, so don't worry about some inconsistencies.

Requirements and constraints

- **React is required.** Beyond that, you are free: any libraries, component approach, styling solution, charting library, or build tooling you like. Use what makes you productive.
- You may use AI tools (Copilot, Claude, etc.). We do this on the job too. We will ask you about your choices, so make sure you understand and can defend what you submit.
- No backend, authentication, persistence, or real API is needed. No tests required (though we will not complain if you write a meaningful one).

What to submit

1. The code, in a Git repository (GitHub link or a zip).
2. A short **README** (half a page is enough) covering:
 - How to run it.
 - What you chose to build and what you deliberately left out, given the time.
 - One or two decisions you are proud of, and one thing you would do differently with more time.
3. We will follow up with a short conversation where you walk us through your choices, especially the UX and product decisions. This is as important to us as the code.

A reminder: thoughtful scope beats breadth. We respect a candidate who builds less but builds it well, and explains why.

Good luck. We are looking forward to seeing how you think.